

/*

*****French*****

Exercice2

On se propose de réaliser un programme de gestion de personnels d’une entreprise. Chaque employé est identifié par son nom, prénom, Département et salaire. Pour pouvoir calculer le salaire d’un employé, deux champs supplémentaires de type double sont nécessaires : base et nbHeures.

- 1) Définir la structure *Employe*.
- 2) Ecrire une fonction *Saisir_Employe* permettant la saisie des informations d’un ensemble d’employés (le nombre d’employés n’est pas connu à priori). La fonction accepte un tableau dynamique de structures en arguments et s’occupe aussi de l’allocation dynamique de mémoire.
- 3) Ecrire une fonction *Informations_Employe* permettant d’afficher les informations de tous les employés (la procédure accepte un tableau dynamique de structures en arguments).
- 4) Ecrire une fonction *Salaire* qui affiche les informations de tous les employés dont le salaire est supérieur à la moyenne.
- 5) Ecrire une fonction *Recherche_Employe* qui permet d’afficher la liste de tous les employés appartenant à un département donné.

Ecrire un programme principal de test.

*****English*****

Exercise2

We propose to carry out a personnel management program for a company. Each employee is identified by their last name, first name, department and salary. To be able to calculate an employee's salary, two additional fields of double type are required: base and nbHours.

- 1) Define the Employee structure.
- 2) Write a function *Saisir_Employe* allowing the entry of information for a set of employees (the number of employees is not known a priori). The function accepts a dynamic array of structures as arguments and also takes care of dynamic memory allocation.
- 3) Write a function *Informations_Employee* allowing to display the information of all the employees (the procedure accepts a dynamic array of structures in arguments).
- 4) Write a Salary function that displays the information of all employees whose salary is higher than the average.
- 5) Write a Search_Employee function that displays the list of all employees belonging to a given department.

Write a main test program.

*/

```

#include <stdio.h>
#include <stdlib.h>
#include <string.h>

Employee *get_employee(Employee *list, int length);
void display_employee(Employee *list, int N);
float average_salary(Employee *e, int length);
void list_employee(Employee *list, int length);
void find_employee_name(Employee *list, int length);

typedef struct employee
{
    char firstName[15];
    char lastName[15];
    char departement[15];
    double hours;
    double basic;
} Employee;

int main()
{
    int length;
    Employee *list;
    printf("\nEnter how many employee: ");
    scanf("%d", &length);
    list = get_employee(list, length);
    display_employee(list, length);
    printf("\t\n***** List Employees Salary *****\n");
    list_employee(list, length);
    find_employee_name(list, length);
}

Employee *get_employee(Employee *list, int length)
{
    int i;
    list = (Employee *)malloc(sizeof(Employee));
    if (list == NULL)
        printf("Allocation failed!\n");

```

```

else
{
    for (i = 0; i < length; i++)
    {
        printf("\n\t***** For Employee %d *****\n", i + 1);
        printf("Enter employee %d Last Name: ", i + 1);
        scanf("%s", &(list + i)->lastName);
        printf("Enter employee %d First Name: ", i + 1);
        scanf("%s", &(list + i)->firstName);
        printf("Enter employee %d Departement: ", i + 1);
        scanf("%s", &(list + i)->departement);
        printf("Enter employee %d Hours: ", i + 1);
        scanf("%lf", &(list + i)->hours);
        printf("Enter employee %d Basic Salary: ", i + 1);
        scanf("%lf", &(list + i)->basic);
    }
}
return list;
}

```

```

}

```

```

void display_employee(Employee *list, int N)
{
    int i;
    for (i = 0; i < N; i++)
    {
        printf("\n\n\t***** For Employee %d *****\n", i + 1);
        printf("\tLast Name: %s\n", (list + i)->firstName);
        printf("\tFirst Name: %s\n", (list + i)->lastName);
        printf("\tdepartement: %s\n", (list + i)->departement);
        printf("\tSalaire: %2f\n", ((list + i)->hours) * ((list + i)->basic));
    }
}

```

```

}

```

```

float average_salary(Employee *e, int length)
{
    int i;
    float sum = 0;
    for (i = 0; i < length; i++)
        sum += (e[i].hours) * (e[i].basic);
}

```

```
    return sum / length;
```

```
}
```

```
void list_employee(Employee *list, int length)
```

```
{
```

```
    int i;
```

```
    for (i = 0; i < length; i++)
```

```
    {
```

```
        if ((list[i].hours) * (list[i].basic) > average_salary((list + i), length))
```

```
            printf("%s %s\n", list[i].firstName, list[i].lastName);
```

```
    }
```

```
}
```

```
void find_employee_name(Employee *list, int length)
```

```
{
```

```
    int i, OK = 0;
```

```
    char Nom[10];
```

```
    printf("Enter the name to search on: ");
```

```
    scanf("%s", Nom);
```

```
    for (i = 0; i < length; i++)
```

```
    {
```

```
        if (strcmp(list[i].lastName, Nom) == 0)
```

```
        {
```

```
            printf("%s %s\n", list[i].lastName, list[i].lastName);
```

```
            OK = 1;
```

```
        }
```

```
    }
```

```
    if (!OK)
```

```
        printf("Employee not found\n");
```

```
}
```

/*

Edit by: MOHAMMED ELIDRISSI LAOUKILI

#idriCoding | #idriLogic | #idri

2021/2022

لا اله الا الله

*/